

Relatório de Auditoria de Segurança

AstrumISP

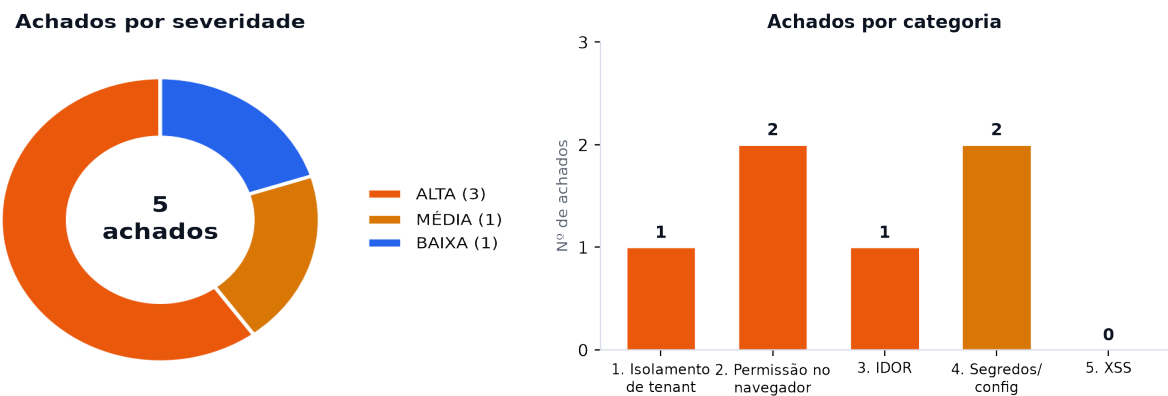
Data: 2026-09-01

Escopo auditado. Monorepo AstrumISP — backend de produção apps/api (Fastify + DDD, ~120 arquivos de rota), frontend legado src/pages (React/Vite), pacotes packages/* e arquivos de deploy (docker-compose, infra/, scripts/, .env.example). Banco único Supabase (Postgres) acessado via service_role; Redis; Qdrant.

Nota metodológica. Cada categoria foi mapeada para a stack detectada: (1) Isolamento de inquilino = filtro manual por tenant_id nos handlers, já que o backend usa supabaseAdmin/service_role e BYPASSA a RLS do Postgres; (2) Permissão no navegador = cruzamento dos gates de papel do React (isAstrum/currentUserRole) com requirePermission no Fastify; (3) IDOR = varredura de todos os handlers com parâmetro de id (path/query/body); (4) Segredos = inspeção de código, configs, docker-compose, CI, scripts e histórico git; (5) XSS = busca por dangerouslySetInnerHTML/innerHTML/eval e renderização de dados em href/src no frontend.

Resumo executivo

Foram identificados **5 achados**: **3 de severidade alta**, **1 média** e **1 baixa**. Nenhum achado crítico. Os riscos centrais concentram-se em **controle de acesso**: um IDOR cross-tenant destrutivo na biometria de voz, aceitação de token de assinante em canais WebSocket de operador, e rotas de configuração privilegiada sem RBAC no servidor. A base é madura (auditorias AUTH-01/05, MT-02 já aplicadas) e o isolamento por tenant é consistente na maioria das rotas — os furos são pontos específicos que escaparam ao padrão.



1	ALTA	Isolamento + IDOR	IDOR e quebra de isolamento de tenant no consentimento de biometria de voz
2	ALTA	Permissão/navegador	WebSocket aceita token de assinante em canais de operador (autorização insuficiente)
3	ALTA	Permissão/navegador	Rotas de configuração privilegiada sem verificação de papel no servidor
4	MÉDIA	Segredos/config	Webhook Meta valida assinatura de forma condicional (fail-open sem FACEBOOK_APP_SECRET)
5	BAIXA	Segredos/config	Defaults inseguros de configuração sem validação fail-closed universal

Pontos fortes (o que está protegido)

- **Segredos fora do versionamento.** `.env` não é rastreado pelo git (apenas `.env.example` com placeholders); nenhum segredo real encontrado no histórico. `docker-compose.yml` usa apenas interpolação `${VAR}`.
- **Isolamento de tenant consistente.** A esmagadora maioria das rotas aplica `.eq('tenant_id', tenantId)` ou repassa o `tenantId` ao serviço: `customers`, `team`, `notifications`, `departments`, `documents`, `hsm-templates`, `negotiation`, `settings` (dados), `voice/calls`, `network-twin`, `campaigns`, `subscriber-twin`, `foundry`, `playbook`, `cobrai-dispatch`.
- **Autenticação endurecida.** O decorator `authenticate` (`server.ts`) bloqueia token de assinante e exige iss/aud de operador (AUTH-01/AUTH-05); `JWT_SECRET < 32 chars` aborta o boot.
- **RBAC presente nas rotas críticas.** `requirePermission` aplicado em `team`, `negotiation`, `documents`, `hsm-templates`, `campaigns`; gate `super_admin` correto em `super-admin.routes` (só `super_admin` passa em `reports:admin`).
- **Webhooks defensivos.** HMAC valida contra `rawBody` cru com `timingSafeEqual` e `fail-closed` (`webhook-hmac.plugin`); Asaas usa token estático em comparação `timing-safe fail-closed`.
- **Regras de negócio anti-abuso.** `negotiation` recomputa valores no servidor e exige auditoria imutável `fail-closed` antes de afrouxar alçada; `/security/unmask` nega o reveal se a auditoria não gravar.
- **Superfície de XSS mínima.** Sem `dangerouslySetInnerHTML` com dado de usuário; `innerHTML` só em templates estáticos (`mapa/marcadores`); URLs de anexo passam por assinatura server-side (`useSignedMediaUrls`). Categoria 5 sem achado explorável.
- **Camadas de defesa web.** helmet com CSP, CORS por allowlist, `lockout` anti brute-force no portal, `rate-limit` e idempotência registrados globalmente.

Pontos fracos (riscos centrais)

- ▲ O isolamento depende de filtro MANUAL por `tenant_id` (`service_role` bypassa a RLS) — qualquer handler que esquecer o filtro vaza entre tenants, como ocorreu em `voice-consent`.
- ▲ Autorização por papel é aplicada rota a rota; onde falta (`WS`, `settings`, `ai-config`, `integration-keys`) a permissão fica só no frontend.
- ▲ Validação de assinatura de webhook condicionada a `env` presente (Meta) — `fail-open` silencioso quando não configurada.

Achados detalhados por categoria

Categoria 1 — Banco sem tranca (isolamento de inquilino)

Sev.	Arquivo:linha	Descrição
ALTA	apps/api/src/domain/ia/voice-consent.routes.ts:23-35 apps/api/src/domain/atendimento/voice-verify.service.ts:14-51	#1. IDOR e quebra de isolamento de tenant no consentimento de biometria de voz

Categoria 2 — Permissão definida no navegador

Sev.	Arquivo:linha	Descrição
ALTA	apps/api/src/domain/realtime/websocket.routes.ts:233-246 apps/api/src/domain/realtime/websocket.routes.ts:51-146	#2. WebSocket aceita token de assinante em canais de operador (autorização insuficiente)
ALTA	apps/api/src/domain/provedor/integration-secrets.routes.ts:24-52 apps/api/src/domain/provedor/settings-page.routes.ts:57-298 (SSO :170, modules, theme, vector-store, embedding) apps/api/src/domain/provedor/ai-config.routes.ts:35-53 src/pages/SettingsPage.tsx:52 (gate de UI: isAstrum = currentUserRole === 'admin')	#3. Rotas de configuração privilegiada sem verificação de papel no servidor

Categoria 3 — IDOR (acesso a objeto por id sem posse)

Sev.	Arquivo:linha	Descrição
ALTA	apps/api/src/domain/ia/voice-consent.routes.ts:23-35 apps/api/src/domain/atendimento/voice-verify.service.ts:14-51	#1. IDOR e quebra de isolamento de tenant no consentimento de biometria de voz

Categoria 4 — Chaves expostas / defaults de segredo

Sev.	Arquivo:linha	Descrição
MÉDIA	apps/api/src/adapters/meta/meta-webhook.routes.ts:104-107 apps/api/src/infrastructure/security/webhook-hmac.plugin.ts:10-17 .env.example:70 (FACEBOOK_APP_SECRET vazio por padrão)	#4. Webhook Meta valida assinatura de forma condicional (fail-open sem FACEBOOK_APP_SECRET)
BAIXA	apps/api/src/infrastructure/database/supabase.client.ts:5-18 apps/api/src/infrastructure/config/env.validator.ts:188-196 infra/sql/create-zep-user.sql:6	#5. Defaults inseguros de configuração sem validação fail-closed universal

Categoria 5 — Inputs sem tratamento (XSS)

Não aplicável de forma explorável nesta stack: não há dangerouslySetInnerHTML/v-html com dado de usuário nem renderização de markdown sem sanitização; innerHTML aparece apenas em templates estáticos (marcadores de mapa) e URLs de anexo passam por assinatura server-side. Ver pontos fortes.

Detalhamento técnico dos achados

ALTA

Achado #1 — IDOR e quebra de isolamento de tenant no consentimento de biometria de voz

Categorias: Isolamento, IDOR**Local:** apps/api/src/domain/ia/voice-consent.routes.ts:23-35; apps/api/src/domain/atendimento/voice-verify.service.ts:14-51

```
app.delete('/api/v2/ia/voice/consent/:customerId', async (req, reply) => {
  const customerId = (req.params as any).customerId;
  await revokeConsent(customerId); // <- sem tenantId
});
// voice-verify.service.ts
export async function revokeConsent(customerId: string) {
  await supabaseAdmin.from('voice_biometry_consents')
    .update({ revoked_at: ... }).eq('customer_id', customerId); // sem tenant_id
  await supabaseAdmin.from('voice_prints').delete()
    .eq('customer_id', customerId); // DELETE cross-tenant
}
```

Por que é explorável. O mecanismo de isolamento do projeto é o filtro manual por `tenant_id` no código (o backend usa `supabaseAdmin` com `service_role`, que BYPASSA a RLS do Postgres). Os handlers GET e DELETE de consentimento recebem `customerId` direto do path e chamam `hasConsent(customerId)/revokeConsent(customerId)` — nenhum dos dois recebe ou aplica `tenant_id`. As funções do service filtram apenas por `customer_id`. O POST, para comparação, exige e aplica `tenantId` corretamente. Qualquer operador autenticado (de qualquer tenant) pode consultar/alterar o consentimento de um cliente de OUTRO provedor apenas informando o `customerId` (UUID).

Impacto. Leitura cross-tenant do status de consentimento LGPD de qualquer cliente e, no DELETE, revogação do consentimento + EXCLUSÃO das `voice_prints` (dado biométrico) de clientes de outros tenants — operação destrutiva e irreversível entre inquilinos.

Correção sugerida. Exigir `tenantId` do JWT nos três handlers e propagá-lo ao service; alterar as assinaturas para `hasConsent(customerId, tenantId)` e `revokeConsent(customerId, tenantId)`, adicionando `.eq('tenant_id', tenantId)` em TODAS as queries (`voice_biometry_consents` e `voice_prints`). Idealmente validar posse (`canAccessResource`) antes de operar.

ALTA

Achado #2 — WebSocket aceita token de assinante em canais de operador (autorização insuficiente)

Categorias: Permissão/navegador**Local:** apps/api/src/domain/realtime/websocket.routes.ts:233-246; apps/api/src/domain/realtime/websocket.routes.ts:51-146

```
async function wsAuthenticate(request, reply) {
  const token = request.headers.authorization?.replace('Bearer ', '') ?? request.query.token;
  if (!token) return reply.status(401)...;
  const payload = request.server.jwt.verify(token); // só valida ASSINATURA
  request.user = payload; // sem checar aud/role
}
// /ws/conversations/:id e /ws/notifications não checam role
```

Por que é explorável. O decorator HTTP `authenticate` (server.ts:90-97) já bloqueia token de assinante e exige iss='astrum-api' + aud='astrum-operator' (correções AUTH-01/AUTH-05). Mas as rotas WebSocket usam um verificador próprio, wsAuthenticate, que apenas valida a assinatura do JWT. Como o token do portal do assinante (role:'subscriber', aud:'subscriber-portal') é assinado com o MESMO JWT_SECRET, ele passa em wsAuthenticate. /ws/conversations/:id e /ws/notifications não fazem checagem de role (só /ws/operator-panel checa). O canal usado é prefixado pelo tenantId do próprio token.

Impacto. Um assinante (cliente final), com seu token de 24h do portal, conecta em /ws/notifications e recebe as notificações operacionais do ISP (ex.: payment_received com invoiceld e valor de TODOS os clientes) e em /ws/conversations/:id lê mensagens ao vivo de QUALQUER conversa do provedor iterando conversationId. É exatamente a classe de furo que a AUTH-01 fechou no REST (inbox de operador), reaberta no WS.

Correção sugerida. Reaproveitar a mesma verificação do decorator `authenticate` no wsAuthenticate: rejeitar role==='subscriber'/aud==='subscriber-portal' e exigir iss='astrum-api' + aud='astrum-operator'. Aplicar checagem de role mínima também em /ws/conversations e /ws/notifications. Evitar token em query string (fica em logs); preferir subprotocolo/header.

ALTA**Achado #3 — Rotas de configuração privilegiada sem verificação de papel no servidor****Categorias:** Permissão/navegador**Local:** apps/api/src/domain/provedor/integration-secrets.routes.ts:24-52;

apps/api/src/domain/provedor/settings-page.routes.ts:57-298 (SSO :170, modules, theme, vector-store, embedding);

apps/api/src/domain/provedor/ai-config.routes.ts:35-53; src/pages/SettingsPage.tsx:52 (gate de UI: isAstrum = currentUserRole === 'admin')

```
// integration-secrets.routes.ts – grava BYOK (OpenAI/Evolution) cifrado
app.put('/api/v2/settings/integration-keys', { onRequest: auth }, ...) // só authenticate
// settings-page.routes.ts
app.put('/api/v2/settings/sso', { onRequest: auth }, ...) // só authenticate
// ai-config.routes.ts
app.put('/api/v2/ai-config/cobrai-settings', { onRequest: auth }, ...) // só authenticate
// Frontend esconde por papel:
const isAstrum = currentUserRole === 'admin'; // SettingsPage.tsx:52
```

Por que é explorável. O RBAC do projeto (requirePermission) é aplicado corretamente em rotas equivalentes (team-page usa users:write; negotiation usa billing:write; documents usa ai_config:write). Mas estas rotas de configuração do provedor têm apenas `onRequest: authenticate` — sem requirePermission. O frontend esconde a tela de Settings por papel (isAstrum === 'admin'), então a permissão só existe no navegador. Qualquer usuário autenticado do tenant (viewer/operator) pode chamar os endpoints diretamente.

Impacto. Escalonamento de privilégio dentro do tenant: um viewer/operator pode (a) sobrescrever as chaves de integração BYOK (OpenAI/Evolution) — DoS da IA/WhatsApp ou desvio para chaves do atacante; (b) alterar o domínio de SSO (settings/sso) — risco de tomada de conta se o login por SSO confiar no domínio; (c) mudar cadência/limites da régua CobrAI, tema, vector-store e módulos habilitados.

Correção sugerida. Adicionar preHandler requirePermission ao gate de escrita destas rotas (ex.: ai_config:write para ai-config; um recurso 'settings'/users' com ação admin para settings e integration-keys). Modelar 'settings' no ROLE_PERMISSIONS. Nunca depender do gate de UI para autorização.

MÉDIA**Achado #4 — Webhook Meta valida assinatura de forma condicional (fail-open sem FACEBOOK_APP_SECRET)****Categorias:** Segredos/config**Local:** apps/api/src/adapters/meta/meta-webhook.routes.ts:104-107;

apps/api/src/infrastructure/security/webhook-hmac.plugin.ts:10-17; .env.example:70 (FACEBOOK_APP_SECRET vazio por padrão)

```
if (process.env.FACEBOOK_APP_SECRET &&
(!rawBody || !validateWebhookSignature(rawBody, signature, 'facebook')) {
  return reply.code(401).send({ code: 'INVALID_SIGNATURE' });
}
// Se FACEBOOK_APP_SECRET NÃO estiver setado, o if inteiro é pulado -> processa sem validar.
// /api/v2/webhook/meta também não está no mapa do webhook-hmac.plugin (só /webhook/facebook).
```

Por que é explorável. A verificação de assinatura do webhook Meta só ocorre quando FACEBOOK_APP_SECRET está definido. No .env.example essa variável vem VAZIA por padrão e é opcional no validador de env, então em um ambiente onde não foi configurada a checagem é totalmente pulada (fail-open). Além disso, a rota /api/v2/webhook/meta não consta no WEBHOOK_ROUTES do plugin HMAC global — a única defesa é o if condicional do handler.

Impacto. Quando a env não está configurada, um atacante pode injetar mensagens de entrada forjadas em qualquer tenant (resolvido por pageld) — disparando o pipeline de IA (custo, spam, poluição de conversas). Explorabilidade condicionada a: canal Meta ativo e FACEBOOK_APP_SECRET ausente (o default).

Correção sugerida. Tornar a validação fail-closed: se o canal Meta estiver habilitado, exigir FACEBOOK_APP_SECRET e rejeitar (401) qualquer POST sem assinatura válida, independentemente da env estar setada. Incluir /api/v2/webhook/meta no mapa do webhook-hmac.plugin ou centralizar a checagem.

BAIXA

Achado #5 — Defaults inseguros de configuração sem validação fail-closed universal**Categorias:** Segredos/config**Local:** apps/api/src/infrastructure/database/supabase.client.ts:5-18; apps/api/src/infrastructure/config/env.validator.ts:188-196; infra/sql/create-zep-user.sql:6

```
const supabaseAnonKey = process.env.SUPABASE_ANON_KEY || ... || 'placeholder';
const supabaseServiceRoleKey = process.env.SUPABASE_SERVICE_ROLE_KEY || ... ||
'placeholder_service_role_key';
if (supabaseServiceRoleKey === 'placeholder_service_role_key') console.warn(...); // só AVISA
// env.validator: fora de 'produção-like', env inválida degrada-open
// create-zep-user.sql: CREATE USER zep_user WITH PASSWORD 'SENHA_FORTE_AQUI';
```

Por que é explorável. Não há segredos reais commitados (o .env não é versionado; .env.example e docker-compose usam placeholders/\${VAR}). Porém há defaults que só emitem warn (supabase.client) e o validador de env degrada-open fora de ambiente 'produção-like'. O script SQL do Zep traz uma senha placeholder literal que vira credencial fraca real se executado sem edição.

Impacto. Baixo em produção (o env.validator faz fail-fast em produção-like e o JWT_SECRET tem fail-fast). Risco de operar degradado em ambientes mal rotulados (staging/preview) e de credencial fraca se o script SQL for rodado verbatim. Endurecimento defensivo, não vazamento direto.

Correção sugerida. Rejeitar no boot valores placeholder de chaves Supabase (fail-closed) em qualquer ambiente não-local; remover a senha placeholder do SQL (exigir via variável) e documentar geração via generate-secrets.sh.

Recomendações priorizadas

P1	Corrigir o IDOR de voice-consent (Achado 1): tenant_id em hasConsent/revokeConsent e no DELETE de voice_prints. Impacto cross-tenant destrutivo.
P1	Fechar o WS (Achado 2): aplicar a mesma verificação de aud/role do decorator authenticate no wsAuthenticate e exigir papel de operador nos canais.
P1	Aplicar RBAC no servidor nas rotas de configuração (Achado 3): integration-keys, settings/*, ai-config/*.
P2	Tornar a validação do webhook Meta fail-closed (Achado 4) e cobri-la pelo HMAC central.
P2	Adicionar um guard de CI (STRICT_TENANT_GUARD/lint) que falhe o build quando uma query tenant-scoped não filtra por tenant_id, prevenindo regressões do tipo Achado 1.
P3	Endurecer defaults (Achado 5): rejeitar placeholders de Supabase no boot fora de dev; remover senha literal do SQL do Zep.
P3	Adicionar guard de esquema (bloquear javascript:) em href/src derivados de dados, como reforço defensivo em ChatPage/PortalPage.

Issues para o GitHub

Texto pronto para copiar e colar. Cada issue está delimitada por marcadores `--- ISSUE n ---` / `--- FIM ISSUE n ---`.

--- ISSUE 1 ---

Título: [Segurança] IDOR e quebra de isolamento de tenant no consentimento de biometria de voz

Labels: security, alta

Problema

O mecanismo de isolamento do projeto é o filtro manual por `tenant_id` no código (o backend usa `supabaseAdmin` com `service_role`, que BYPASSA a RLS do Postgres). Os handlers GET e DELETE de consentimento recebem `customerId` direto do path e chamam `hasConsent(customerId)/revokeConsent(customerId)` – nenhum dos dois recebe ou aplica `tenant_id`. As funções do `service` filtram apenas por `customer_id`. O POST, para comparação, exige e aplica `tenantId` corretamente. Qualquer operador autenticado (de qualquer tenant) pode consultar/alterar o consentimento de um cliente de OUTRO provedor apenas informando o `customerId` (UUID).

Evidência

```
- `apps/api/src/domain/ia/voice-consent.routes.ts:23-35`
- `apps/api/src/domain/atendimento/voice-verify.service.ts:14-51`
```

```
``ts
app.delete('/api/v2/ia/voice/consent/:customerId', async (req, reply) => {
  const customerId = (req.params as any).customerId;
  await revokeConsent(customerId); // <- sem tenantId
});
// voice-verify.service.ts
export async function revokeConsent(customerId: string) {
  await supabaseAdmin.from('voice_biometry_consent')
    .update({ revoked_at: ... }).eq('customer_id', customerId); // sem tenant_id
  await supabaseAdmin.from('voice_prints').delete()
    .eq('customer_id', customerId); // DELETE cross-tenant
}
``
```

Impacto

Leitura cross-tenant do status de consentimento LGPD de qualquer cliente e, no DELETE, revogação do consentimento + EXCLUSÃO das `voice_prints` (dado biométrico) de clientes de outros tenants – operação destrutiva e irreversível entre inquilinos.

Correção sugerida

Exigir `tenantId` do JWT nos três handlers e propagá-lo ao `service`; alterar as assinaturas para `hasConsent(customerId, tenantId)` e `revokeConsent(customerId, tenantId)`, adicionando `.eq('tenant_id', tenantId)` em TODAS as queries (`voice_biometry_consent` e `voice_prints`). Idealmente validar posse (`canAccessResource`) antes de operar.

Critérios de aceite

- [] GET/DELETE `/api/v2/ia/voice/consent/:customerId` retornam 404/nada quando o `customer` pertence a outro tenant
- [] `revokeConsent` e `hasConsent` recebem `tenantId` e aplicam `.eq('tenant_id')` em toda query
- [] DELETE de `voice_prints` filtra por `customer_id` E `tenant_id`
- [] Teste Vitest cobrindo tentativa cross-tenant (deve falhar/isolar)

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] WebSocket aceita token de assinante em canais de operador (autorização insuficiente)

Labels: security, alta

Problema

O decorator HTTP ``authenticate`` (`server.ts:90-97`) já bloqueia token de assinante e exige `iss='astrum-api' + aud='astrum-operator'` (correções AUTH-01/AUTH-05). Mas as rotas WebSocket usam um verificador próprio, `wsAuthenticate`, que apenas valida a assinatura do JWT. Como o token do portal do assinante (`role:'subscriber', aud:'subscriber-portal'`) é assinado com o MESMO `JWT_SECRET`, ele passa em

wsAuthenticate. /ws/conversations/:id e /ws/notifications não fazem checagem de role (só /ws/operator-panel checa). O canal usado é prefixado pelo tenantId do próprio token.

Evidência

```
- `apps/api/src/domain/realtime/websocket.routes.ts:233-246`
- `apps/api/src/domain/realtime/websocket.routes.ts:51-146`
```

```
```ts
```

```
async function wsAuthenticate(request, reply) {
 const token = request.headers.authorization?.replace('Bearer ', '') ?? request.query.token;
 if (!token) return reply.status(401)...;
 const payload = request.server.jwt.verify(token); // só valida ASSINATURA
 request.user = payload; // sem checar aud/role
}
// /ws/conversations/:id e /ws/notifications não checam role
```
```

Impacto

Um assinante (cliente final), com seu token de 24h do portal, conecta em /ws/notifications e recebe as notificações operacionais do ISP (ex.: payment_received com invoiceId e valor de TODOS os clientes) e em /ws/conversations/:id lê mensagens ao vivo de QUALQUER conversa do provedor iterando conversationId. É exatamente a classe de furo que a AUTH-01 fechou no REST (inbox de operador), reaberta no WS.

Correção sugerida

Reaproveitar a mesma verificação do decorator `authenticate` no wsAuthenticate: rejeitar role==='subscriber'/aud==='subscriber-portal' e exigir iss='astrum-api' + aud='astrum-operator'. Aplicar checagem de role mínima também em /ws/conversations e /ws/notifications. Evitar token em query string (fica em logs); preferir subprotocolo/header.

Critérios de aceite

```
- [ ] wsAuthenticate rejeita tokens com aud='subscriber-portal' ou role='subscriber' (403)
- [ ] wsAuthenticate exige iss='astrum-api' e aud='astrum-operator'
- [ ] /ws/conversations/:id e /ws/notifications exigem papel de operador+
- [ ] Teste conectando com token de assinante retorna 401/403
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Rotas de configuração privilegiada sem verificação de papel no servidor
Labels: security, alta

Problema

O RBAC do projeto (requirePermission) é aplicado corretamente em rotas equivalentes (team-page usa users:write; negotiation usa billing:write; documents usa ai_config:write). Mas estas rotas de configuração do provedor têm apenas `onRequest: authenticate` – sem requirePermission. O frontend esconde a tela de Settings por papel (isAstrum === 'admin'), então a permissão só existe no navegador. Qualquer usuário autenticado do tenant (viewer/operador) pode chamar os endpoints diretamente.

Evidência

```
- `apps/api/src/domain/provedor/integration-secrets.routes.ts:24-52`
- `apps/api/src/domain/provedor/settings-page.routes.ts:57-298 (SSO :170, modules, theme, vector-store, embedding)`
- `apps/api/src/domain/provedor/ai-config.routes.ts:35-53`
- `src/pages/SettingsPage.tsx:52 (gate de UI: isAstrum = currentUserRole === 'admin')`
```

```
```ts
```

```
// integration-secrets.routes.ts – grava BYOK (OpenAI/Evolution) cifrado
app.put('/api/v2/settings/integration-keys', { onRequest: auth }, ...) // só authenticate
// settings-page.routes.ts
app.put('/api/v2/settings/sso', { onRequest: auth }, ...) // só authenticate
// ai-config.routes.ts
app.put('/api/v2/ai-config/cobrai-settings', { onRequest: auth }, ...) // só authenticate
// Frontend esconde por papel:
const isAstrum = currentUserRole === 'admin'; // SettingsPage.tsx:52
```
```

Impacto

Escalonamento de privilégio dentro do tenant: um viewer/operator pode (a) sobrescrever as chaves de integração BYOK (OpenAI/Evolution) – DoS da IA/WhatsApp ou desvio para chaves do atacante; (b) alterar o domínio de SSO (settings/sso) – risco de tomada de conta se o login por SSO confiar no domínio; (c) mudar cadência/limites da régua CobrAI, tema, vector-store e módulos habilitados.

Correção sugerida

Adicionar preHandler requirePermission ao gate de escrita destas rotas (ex.: ai_config:write para ai-config; um recurso 'settings'/'users' com ação admin para settings e integration-keys). Modelar 'settings' no ROLE_PERMISSIONS. Nunca depender do gate de UI para autorização.

Critérios de aceite

- [] PUT em /settings/*, /ai-config/* e /settings/integration-keys exigem papel admin+ no servidor
- [] viewer/operator recebem 403 nesses endpoints (teste automatizado)
- [] Recurso de configuração modelado no RBAC (ROLE_PERMISSIONS)
- [] Auditoria registrada em alterações de SSO e chaves de integração

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Webhook Meta valida assinatura de forma condicional (fail-open sem FACEBOOK_APP_SECRET)
Labels: security, média

Problema

A verificação de assinatura do webhook Meta só ocorre quando FACEBOOK_APP_SECRET está definido. No .env.example essa variável vem VAZIA por padrão e é opcional no validador de env, então em um ambiente onde não foi configurada a checagem é totalmente pulada (fail-open). Além disso, a rota /api/v2/webhook/meta não consta no WEBHOOK_ROUTES do plugin HMAC global – a única defesa é o if condicional do handler.

Evidência

- `apps/api/src/adapters/meta/meta-webhook.routes.ts:104-107`
- `apps/api/src/infrastructure/security/webhook-hmac.plugin.ts:10-17`
- `.env.example:70 (FACEBOOK\_APP\_SECRET vazio por padrão)`

```
``ts
if (process.env.FACEBOOK_APP_SECRET &&
(!rawBody || !validateWebhookSignature(rawBody, signature, 'facebook'))) {
return reply.code(401).send({ code: 'INVALID_SIGNATURE' });
}
// Se FACEBOOK_APP_SECRET NÃO estiver setado, o if inteiro é pulado -> processa sem validar.
// /api/v2/webhook/meta também não está no mapa do webhook-hmac.plugin (só /webhook/facebook).
...
```

Impacto

Quando a env não está configurada, um atacante pode injetar mensagens de entrada forjadas em qualquer tenant (resolvido por pageId) – disparando o pipeline de IA (custo, spam, poluição de conversas). Explorabilidade condicionada a: canal Meta ativo e FACEBOOK_APP_SECRET ausente (o default).

Correção sugerida

Tornar a validação fail-closed: se o canal Meta estiver habilitado, exigir FACEBOOK_APP_SECRET e rejeitar (401) qualquer POST sem assinatura válida, independentemente da env estar setada. Incluir /api/v2/webhook/meta no mapa do webhook-hmac.plugin ou centralizar a checagem.

Critérios de aceite

- [] POST /api/v2/webhook/meta sem assinatura válida retorna 401 mesmo sem FACEBOOK_APP_SECRET (fail-closed)
- [] Boot valida presença de FACEBOOK_APP_SECRET quando o canal Meta está habilitado
- [] Rota meta coberta pela verificação HMAC central
- [] Teste com payload forjado é rejeitado

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] Defaults inseguros de configuração sem validação fail-closed universal
Labels: security, baixa

Problema

Não há segredos reais commitados (o .env não é versionado; .env.example e docker-compose usam placeholders/\${VAR}). Porém há defaults que só emitem warn (supabase.client) e o validador de env degrada-open fora de ambiente 'produção-like'. O script SQL do Zep traz uma senha placeholder literal que vira credencial fraca real se executado sem edição.

Evidência

```
- `apps/api/src/infrastructure/database/supabase.client.ts:5-18`  
- `apps/api/src/infrastructure/config/env.validator.ts:188-196`  
- `infra/sql/create-zep-user.sql:6`
```

```
``ts
```

```
const supabaseAnonKey = process.env.SUPABASE_ANON_KEY || ... || 'placeholder';  
const supabaseServiceRoleKey = process.env.SUPABASE_SERVICE_ROLE_KEY || ... ||  
'placeholder_service_role_key';  
if (supabaseServiceRoleKey === 'placeholder_service_role_key') console.warn(...); // só AVISA  
// env.validator: fora de 'produção-like', env inválida degrada-open  
// create-zep-user.sql: CREATE USER zep_user WITH PASSWORD 'SENHA_FORTE_AQUI';  
``
```

Impacto

Baixo em produção (o env.validator faz fail-fast em produção-like e o JWT_SECRET tem fail-fast). Risco de operar degradado em ambientes mal rotulados (staging/preview) e de credencial fraca se o script SQL for rodado verbatim. Endurecimento defensivo, não vazamento direto.

Correção sugerida

Rejeitar no boot valores placeholder de chaves Supabase (fail-closed) em qualquer ambiente não-local; remover a senha placeholder do SQL (exigir via variável) e documentar geração via generate-secrets.sh.

Critérios de aceite

- [] Boot aborta se SUPABASE_SERVICE_ROLE_KEY == 'placeholder_service_role_key' fora de dev
- [] Script create-zep-user.sql não contém senha literal
- [] Documentação aponta generate-secrets.sh como fonte dos segredos

--- FIM ISSUE 5 ---